CSIS 4495 - 071 Progress Report 1

Theseus: Weekly AI Review for Goal-Time Alignment

Team Name: Zisyphuz

Project Name: Theseus: Weekly AI Review for Goal-Time Alignment

Team Members: Dong Zhang; Zhi Kang

Instructor: Michael Ma

Submission Date: June 15, 2026

Current Status: Sprint 0 complete; Sprint 1 starts with backend persistence.

1. Project Update

1.1 Current Position and Scope

Theseus is a weekly AI review system for goal-time alignment. The project direction has stayed stable since the proposal. The system helps users review whether their real weekly time supported their important goals. The focus is the completed week, not automatic scheduling.

The MVP remains limited to one review loop: goals, weekly plan, actual time logs, activity-impact labels, optional reflection, evidence checks, weekly review, and one realistic next-week adjustment. This scope follows the proposal style: Theseus is a review layer, not a calendar replacement, task manager, wearable-dependent product, mobile sync tool, or mental health diagnosis system.

This wording is important because the team has a working local review demo, but not a fully integrated product yet. The current report therefore separates completed Sprint 0 foundation work from planned Sprint 1 persistence work.

Area	Status	Meaning for Progress Report 1
Project direction	Stable	The problem, target users, MVP boundary, and review-loop idea remain the same as the proposal.
Sprint 0 foundation	Complete	Repository structure, core documents, sample data, backend skeleton, and local demo are ready for further build work.
Sprint 1 focus	Next	The next stage will move from sample-file review to SQLite persistence, CRUD APIs, and stored weekly review results.

1.2 Tasks Completed Since the Proposal

Since the proposal, the team has completed Sprint 0. The result is not a finished product, but it gives the project a buildable and trackable base for Sprint 1. The main work completed is listed below.

- Created a GitHub repository structure with backend, frontend, review_engine, docs, data, and evaluation folders. https://github.com/dongzhang2077/theseus-weekly-review
- Prepared project documents, including requirements, architecture notes, data model, API contract, agile delivery plan, backlog, decision log, and GitHub workflow notes.
- Built a FastAPI backend skeleton with /health and /reviews/weekly/analyze routes.
- Defined Pydantic schemas for goals, projects, weekly plans, planned items, time logs, daily reflections, and weekly review requests.
- Added sample weekly data at data/sample/sample_week.json and a local script to run a sample review.
- Prepared a review quality rubric so later outputs can be checked for accuracy, supportiveness, realism, and evidence support.

1.3 Current Implementation Status and Demo Evidence

The current technical evidence is a local review-engine demo. The sample data can be loaded and passed through review_engine.rules.analyze_week. The output contains wins, insights, risk flags,

next steps, evidence, and generated text. This proves that the basic review logic can produce a structured weekly review from one sample week.

The demo path is currently: sample_week.json -> review_engine -> structured weekly review. It does not yet prove database persistence or frontend integration. The team will not describe SQLite storage, full CRUD flows, or a connected frontend review page as completed work until they are implemented and verified.

Current Item	Progress	Evidence or Boundary
Backend skeleton	Exists	FastAPI application and basic analyze route are in place.
Local review demo	Working locally	scripts/run_sample_review.py can analyze the sample week and return structured output.
SQLite persistence	Planned for Sprint 1	The current demo does not store records in a database yet.
CRUD APIs	Planned for Sprint 1	Entity creation, update, and retrieval APIs still need implementation and verification.
Frontend review page	Planned later	The page should be connected after the backend data path becomes stable.

1.4 Challenges, Assumptions, and Actions Taken

The main challenge after the proposal was scope control. Theseus could easily become too large if scheduling, mobile sync, wearable data, and AI review were built at the same time. The team therefore kept Sprint 1 focused on backend persistence and the weekly review data path.

Another challenge is review quality. Raw AI advice can become vague if it is not tied to clear evidence. The current design uses deterministic evidence checks first. AI wording or LLM polish can be added later, but it should explain the evidence instead of replacing it. The feedback tone also needs control, because a review that only lists missed tasks can feel blaming. For this reason, the report and demo keep the win -> insight -> next step structure.

Risk or Assumption	Action Taken
Implementation size may grow too fast.	Keep Sprint 1 limited to SQLite schema, repository layer, CRUD APIs, sample loader, and stored review path.
AI feedback may be too general.	Use rule-based evidence checks before generated wording.
Input logs may be incomplete.	Start with clean sample weeks and validation rules before user testing.
Feedback tone may feel negative.	Use positive recognition and one realistic next step instead of a long task dump.

1.5 Planned Tasks for the Next Stage

Sprint 1 starts after Progress Report 1 and focuses on backend persistence. The target is to move from a sample-file path to a stored review path: sample_week.json -> SQLite -> review_engine -> stored weekly_review.

- Implement SQLite tables for goals, projects, activities, weekly plans, planned items, time logs, daily reflections, and weekly reviews.
- Add a repository layer so SQL logic does not stay inside FastAPI route handlers.
- Implement CRUD APIs for Sprint 1 entities and verify them with sample data.
- Build a sample data loader that imports sample_week.json into SQLite.
- Connect database-loaded records to the review engine and store weekly review results.
- Prepare a verification script for the sample data -> stored review path.

Phase	Focus	Main Deliverable
Sprint 0	Foundation complete	Proposal submitted; MVP documented; repository, docs, sample data, and local demo prepared.
Sprint 1	Backend persistence	SQLite schema, repository layer, CRUD APIs, sample loader, and stored weekly review path.
Sprint 2	Review hardening and frontend preparation	More sample weeks, stronger rules, output polish, and frontend preparation.
Sprint 3-4	Prototype, evaluation, and polish	Frontend review page, rubric testing, classmate feedback, revisions, and final defense support.

1.6 Evaluation Plan

The evaluation will test MVP review quality, not long-term behavior change. The team plans to prepare 3-5 sample weekly records and run them through the review engine. Each output will be checked for factual accuracy, goal relevance, positive recognition, actionability, restraint, risk detection, and evidence support. The team also plans to collect limited feedback from 2-3 classmates and revise the rules and wording based on the findings.

No major scope change is planned. The timeline is still aligned with the course schedule, but the next sprint has been made more concrete: backend persistence comes before frontend integration.

References

Anthropic. (n.d.). Claude. Retrieved June 15, 2026, from https://www.anthropic.com/claude

FastAPI. (n.d.). FastAPI documentation. Retrieved June 15, 2026, from https://fastapi.tiangolo.com/

GitHub Docs. (n.d.). GitHub Docs. Retrieved June 15, 2026, from https://docs.github.com/

Google. (n.d.). Gemini. Retrieved June 15, 2026, from https://gemini.google.com/

OpenAI. (n.d.). ChatGPT. Retrieved June 15, 2026, from https://chatgpt.com/

Pydantic. (n.d.). Pydantic documentation. Retrieved June 15, 2026, from https://docs.pydantic.dev/

Python Software Foundation. (n.d.). Python documentation. Retrieved June 15, 2026, from https://docs.python.org/3/

SQLite. (n.d.). SQLite documentation. Retrieved June 15, 2026, from https://www.sqlite.org/docs.html

Appendix A: AI Usage Report and Documentation

AI tools were used as support, not as final authority. AI supported project planning, structural drafting, code skeleton discussion, rules-engine review, slide refinement, speaker-script preparation, and progress-report wording. Final project scope, implementation claims, repository status, and submission decisions were checked by the team.

A.1 AI Tools and Main Uses

Tool / Channel	Main Use	Human Control
Gemini Flash / Pro variants via Google Gemini	Project planning, structural drafting of markdown documents, Pydantic schema ideas, and slide wording refinement.	The team selected useful suggestions and revised them to match the actual MVP scope.
ChatGPT / document assistant	Progress report restructuring, template compliance review, plain, concise academic wording, and style consistency with the earlier proposal.	The team checked the final content, page structure, and whether technical claims were accurate.

A.2 Representative AI-Assisted Work

Area	Prompt or Instruction Summary	Output Used and Verification
Scope review	Analyze the Theseus proposal, extract core requirements, define a strict MVP boundary, and outline a modular architecture.	Helped form the project charter and product requirements. The team removed calendar automation, wearable dependency, and mobile sync from the MVP.
Schema design	Generate Pydantic v2 schemas for Goal, Project, WeeklyPlan, PlannedItem, TimeLog, DailyReflection, and WeeklyReviewRequest.	Draft schema ideas were integrated into backend/app/schemas.py after human review and constraint checking.
Rules engine logic	Draft deterministic checks for goal alignment, plan-vs-actual drift, activity mix, slack risk, and dormancy.	Formed part of the rule-engine design. The team tested the local sample review and adjusted rule boundaries.
Report and slide refinement	Review the progress report and presentation materials. Keep the progress update concise and factual for a five-minute class update.	Improved structure, wording, and delivery flow. The team checked that local demo status was not overstated.

A.3 Human Verification

- The rules in review_engine.rules were tested with scripts/run_sample_review.py against data/sample/sample_week.json.
- The team reviewed the wording so SQLite persistence, CRUD APIs, and frontend pages are described as planned work, not completed work.
- The progress report structure was checked against the CSIS 4495 Progress Report 1 template: Project Update, References, and AI Usage Report.